29. Pruebas unitarias

Estacionamiento y cobertura

Jonathan Serna

Ingeniería en Software

Programación Asistida por Inteligencia Artificial

1. Objetivo y especificación

Mi objetivo fue comprobar la función **calculate_toll** con pruebas basadas en su especificación. Primero preparé tres pruebas unitarias, después solicité a una IA las pruebas adicionales y finalmente medí la cobertura. Conservé intacto el archivo **parking_lot.py** recibido.

Contrato proporcionado

```
calculate_toll(minutes: int, type: str = "auto",
               weekend: bool = False)
```

Regla	Resultado esperado
Minutos negativos	Lanzar ValueError.
Primeros 15 minutos	Estancia gratuita hasta el minuto 15.
Hora completa o fracción	Redondear hacia arriba las horas.
Tarifas por hora	Auto: $20. Moto: $10.
Otro tipo de vehículo	Lanzar ValueError.
Fin de semana	Aumentar la tarifa un 50 %.
Máximo por estancia	$150, también con recargo.

Ejemplos: **calculate_toll(90) = 40**; **calculate_toll(16, "moto") = 10**; **calculate_toll(120, "auto", True) = 60**.

2. Tres pruebas unitarias iniciales

Elegí una validación, el límite de gratuidad y un ejemplo de cobro. Cada prueba tiene una expectativa obtenida de la especificación. Las guardé en **test_initial.py**.

```
import unittest

from parking_lot import calculate_toll


class InitialTollTests(unittest.TestCase):
    """Check validation, free parking, and a supplied billing example."""

    def test_negative_minutes_raise_value_error(self):
        """Reject a stay with negative minutes."""
        with self.assertRaises(ValueError):
            calculate_toll(-1)

    def test_first_fifteen_minutes_are_free(self):
        """Charge nothing at the end of the free period."""
        self.assertEqual(calculate_toll(15), 0)

    def test_ninety_minutes_cost_forty_for_auto(self):
        """Round ninety minutes up to two billable hours."""
        self.assertEqual(calculate_toll(90), 40)
```

Caso	Esperado	Resultado
-1 minuto	ValueError	Aprobado
15 minutos	$0	Aprobado
90 minutos de auto	$40	Aprobado

Resultado real: **3 pruebas aprobadas**, código de salida 0. La cobertura inicial fue de **13/19 líneas (68.42 %)**; trace muestra 68 % al usar enteros.

La salida completa está en **evidence/initial/tests.txt**. También recibí asistencia de IA al preparar estas tres pruebas; las distingo de la ampliación posterior solicitada a otro agente.

3. Generación de las demás pruebas con IA

Después de ejecutar las tres pruebas, envié la especificación y sus expectativas a un agente separado. No le compartí la implementación. Así pude comparar el programa con el contrato sin pedir a la IA que imitara el comportamiento del código existente.

Solicitud enviada (extractos literales)

Actúa como generador de pruebas adicionales, independiente de la implementación. Solo devuelve el código y una matriz breve; NO escribas archivos ni leas parking_lot.py ni otros ejercicios.

Genera pruebas adicionales en una sola clase GeneratedTollTests(unittest.TestCase), importando calculate_toll desde parking_lot. Usa un método por caso, expectativas explícitas y breves docstrings.

No pruebes contratos no especificados (float, None, mensajes exactos o tipo exacto del retorno). No adaptes expectativas a ningún código. Evita redundancias innecesarias.

El mensaje completo, incluida la especificación reproducida, está en **docs/ai-request.md**. La respuesta se incorporó en **test_generated.py**, con 31 métodos adicionales y sin cambiar sus expectativas.

Grupo adicional	Casos cubiertos
Duración y redondeo	0, 14, 16, 60, 61, 120 y 121 minutos.
Moto y fin de semana	Tarifas de $10/$20 y recargo del 50 %.
Fronteras del máximo	Antes, al alcanzar y al superar $150 cuando la tarifa permite esos importes.
Estancias largas	Un día en auto y una semana en moto; el máximo es por estancia.
Tipos inválidos	pension a 0, 15 y 60 minutos; truck a 60.

La especificación fue el criterio de aceptación. Mantuve los fallos visibles; no los marqué como esperados ni ajusté las pruebas para que el código aprobara.

4. Resultados y discrepancias

Batería	Total	Pasan	Fallan	Errores
Inicial	3	3	0	0
Adicional de IA	31	21	10	0
Completa	34	24	10	0

La batería completa terminó con código de salida **1**: hubo diez fallos de aserción y ningún error de ejecución. Las tarifas ordinarias, el redondeo, la gratuidad para vehículos válidos y el recargo sí aprobaron los casos incluidos. Estos son todos los casos que fallaron:

Entrada (minutos, tipo, fin de semana)	Esperado	Obtenido
(421, "auto", False)	$150	$160
(481, "auto", False)	$150	$180
(901, "moto", False)	$150	$160
(301, "auto", True)	$150	$180
(601, "moto", True)	$150	$165
(1440, "auto", False)	$150	$200
(10080, "moto", False)	$150	$200
(0, "pension", False)	ValueError	$0
(15, "pension", False)	ValueError	$0
(60, "pension", False)	ValueError	$0

Los siete fallos de importe muestran que **min(total, 200)** usa un máximo de $200. Los tres fallos de validación muestran dos rutas: la gratuidad devuelve cero antes de validar el tipo, y la rama **pension** lo admite en estancias de pago. El contrato solo acepta auto y moto.

Conservé estos defectos para respetar la consigna. Las trazas y los nombres de las diez pruebas están en **evidence/full/tests.txt**.

5. Prueba de cobertura

Medí únicamente la cobertura de líneas de **parking_lot.py** con trace. El porcentaje indica cuántas de sus líneas ejecutables se ejecutaron al menos una vez; incluye la carga del módulo. No medí cobertura de ramas.

Batería	Ejecutadas / total	Sin ejecutar	Cobertura
Tres pruebas iniciales	13 / 19	6	68.42 %
Batería completa	19 / 19	0	100 %

Listado final real de trace

```
   #!/usr/bin/env python3
   # -*- coding: utf-8 -*-
 1: """
   Developer: .
   Creation Date: 2026-10-26.
   Description: Parking lot excercise.
   """

 1: import math


 1: def calculate_toll(minutes: float, type: str = "auto", weekend: bool = False) -> int:
34:     if minutes < 0:
 1:         raise ValueError("Minutes cannot be negative")
33:     if minutes <= 15:
 6:         return 0

27:     hours = math.ceil(minutes / 60)
27:     if type == "auto":
15:         toll = 20
12:     elif type == "moto":
10:         toll = 10
 2:     elif type == "pension":
 1:         return 0
        else:
 1:         raise ValueError("Invalid vehicle type.")
25:     total = hours * toll
25:     if weekend:
 9:         total *= 1.5
25:     return min(total, 200)
```

Los números a la izquierda son conteos de ejecución. El archivo inicial marca con **>>>>>>** las seis líneas que faltaban. En el listado final no queda ninguna de esas marcas. La rama incorrecta de pension también se ejecutó al comprobar que debía rechazarse.

La salida completa está en **evidence/full/coverage-run.txt**. El 100 % corresponde al módulo evaluado; no se usa el porcentaje de los archivos de pruebas para justificar su calidad.

6. Observaciones y reproducción

Párrafo de observaciones

Al ejecutar las pruebas observé que las tres iniciales aprobaron, pero solo cubrieron 13 de las 19 líneas reportadas por trace, equivalentes al 68.42 %. Después de ampliar los casos con IA alcancé el 100 % de cobertura de líneas; aun así, 10 de las 34 pruebas fallaron. Esto me permitió distinguir entre ejecutar todo el código y cumplir la especificación. Los fallos se concentraron en el máximo por estancia, que el programa limita a $200 en lugar de $150, y en la aceptación de pension, incluido el periodo gratuito donde se devuelve cero antes de validar el vehículo. Los límites de minutos y del tope ayudaron a detectar diferencias que los ejemplos habituales no mostraban. Conservé parking_lot.py sin cambios para documentar el resultado real. Como estudiante de Ingeniería en Software, concluyo que necesito revisar tanto la cobertura como las aserciones y sus resultados; además, esta medición no demuestra cobertura de ramas ni prueba entradas fuera del contrato.

Cómo reproducir

Desde la carpeta **29vo Pruebas estacionamiento**, con Python 3.12, puedo ejecutar:

```
python -B -m unittest -v test_initial
python -B -m unittest -v test_initial test_generated

python -B -m trace --count --missing --summary \
  --coverdir evidence/reproduced --module unittest \
  -v test_initial test_generated
```

Las barras inversas anteriores indican continuación visual; en PowerShell uno las líneas en un único comando. El README incluye comandos de una sola línea y un filtro para omitir la biblioteca estándar en el informe. Ejecuto unittest también por separado porque trace captura SystemExit y su código de salida no refleja esos fallos.

Python 3.12: unittest; Python 3.12: trace.